📘 MODULE 31: SPLUNK SOAR (FROM ZERO)

Version note: SOAR was formerly **Phantom**, so you'll see old names in docs and in job posts. Menu names, playbook editor features, and ES integration details change between releases. Verify them against the docs for the version your employer runs. This is the full beginner-to-working-knowledge course. SOAR is a separate product with a separate license, and it is **not** part of Splunk Enterprise.

31.1 Concept: what SOAR is

SOAR = Security Orchestration, Automation, and Response.

Word	Meaning	Example
Orchestration	Connecting many different security tools so they work together	SOAR talks to your firewall, EDR, email gateway, and ticketing system
Automation	Running tasks without a human clicking	Automatically look up an IP's reputation
Response	Taking action on a threat	Block the IP, isolate the host, disable the user

In one sentence: SOAR takes the repetitive steps an analyst does by hand for every alert and runs them automatically, using **playbooks**.

31.2 Why SOAR exists

The problem in a SOC:

- Thousands of alerts per day, too few analysts.
- For each phishing alert, an L1 manually does the same 10 steps: copy the URL, check VirusTotal, check the sender, search mailboxes, open a ticket, and so on. That takes 15-30 minutes each.
- Slow response means more damage, and boring work means burnout and mistakes.

SOAR solves this by:

1. **Speed:** seconds instead of minutes (lower **MTTR**, mean time to respond).
2. **Consistency:** the same steps every time, with no skipped checks.
3. **Scale:** one playbook handles 1,000 alerts.
4. **Documentation:** every action is logged automatically.
5. **Freeing analysts** for real investigation.

31.3 SIEM vs SOAR (very common interview question)

	Splunk (SIEM / ES)	Splunk SOAR
Main job	**Detect:** collect logs and raise alerts	**Respond:** act on alerts
Input	Logs and events	Alerts/events from Splunk and other tools
Output	Notable events	Automated actions and case records
Speaks to	Data sources	Security tools (via APIs)
Analogy	The smoke detector	The sprinkler system plus the fire report

They are complementary: **ES detects, SOAR responds.**

31.4 Core terminology (learn these exactly)

Term	Meaning
Container	The case/ticket-like object in SOAR holding one event or incident. Has a name, label, severity, status, and owner. (The basic unit of work.)
Artifact	A piece of evidence inside a container: an IP, a URL, a file hash, a username, an email address. A container holds many artifacts.
CEF	**Common Event Format** field names used on artifacts: `sourceAddress`, `destinationAddress`, `fileHash`, `requestURL`, `sourceUserName`, `deviceHostname`. SOAR actions know to read these.
Label	A tag on a container (such as `events`, `phishing`) that decides **which playbooks run**.
App	A connector (integration) to an external tool: VirusTotal, Splunk, ServiceNow, Slack, Active Directory, CrowdStrike, firewalls, and others.
Asset	A **configured instance of an app**: the app plus credentials and URL. Example: app `VirusTotal` + your API key = asset `virustotal_prod`.
Action	A single operation an app can do: `ip reputation`, `block ip`, `get file`, `run query`, `disable user`.
Playbook	An automated workflow that runs actions and logic on a container.
Workbook / Tasks	Manual checklists for analysts, used when humans must follow steps (a response plan).
Prompt	A playbook step that **asks a human** to approve or choose before continuing.
Custom function	Reusable code block, written in Python, that playbooks can call.
Ingestion	How events get into SOAR (from Splunk, email, API, polling).

Hierarchy to memorize:

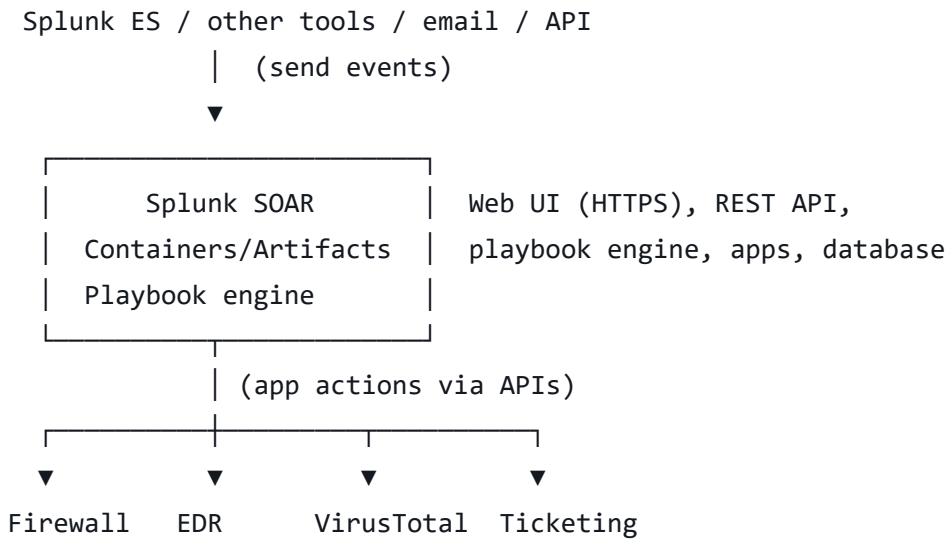
```
SOAR
 ├─ Apps ──► Assets (configured with credentials)
 └─ Containers (events/cases)
       ├─ Artifacts (IPs, URLs, hashes, users)
       ├─ Playbook runs ──► Actions (executed through assets)
       └─ Notes, comments, tasks, files, activity log
```

31.5 Action types

Type	Purpose	Examples
Generic	General operations	`send email`, `create ticket`
Investigate	**Read-only** gathering of information	`ip reputation`, `domain reputation`, `file reputation`, `whois ip`, `get user attributes`
Contain	Stop the threat's spread	`block ip`, `block url`, `quarantine device`, `disable user`
Correct	Undo or repair	`unblock ip`, `restore`, `enable user`

Safety principle: investigate actions are safe to fully automate. **Contain actions often need human approval** (a prompt), because blocking the CEO's IP by mistake is an outage.

31.6 Architecture (on-premises, overview)



- **Splunk SOAR (On-premises):** you install and manage it (typically on supported Linux distributions, or in a container for community use).
- **Splunk SOAR (Cloud):** Splunk hosts it. An **automation broker** (a small component you install inside your network) lets cloud SOAR reach internal tools that aren't exposed to the internet.
- **Community Edition:** a free tier for learning, with usage limits. Registration is required, so check the current terms.

- Web UI uses **HTTPS (port 443)** on the SOAR host. The REST API is on the same host under `/rest/`.

31.7 How Splunk connects to SOAR

Two directions:

A) Splunk → SOAR (send alerts for response)

- Install the **Splunk App for SOAR Export** on the Splunk side and configure a connection to your SOAR server and an **authentication token**.
- In ES (or any alert), use the adaptive/alert action **"Send to SOAR"** (or **"Run playbook in SOAR"**). The notable or alert becomes a **container** in SOAR, and its fields become **artifacts**.
- Mapping decides which Splunk fields (`src_ip`, `user`) become which CEF fields.

B) SOAR → Splunk (query and enrich)

- The **Splunk app in SOAR** (with an asset pointing to your Splunk) lets playbooks **run SPL queries**, return results, and post events (for example, `run query`).
- A playbook can search Splunk for "other activity from this IP" and add the results to the case.

Interview line: "ES creates a notable, sends it to SOAR as a container, a playbook enriches and responds, and SOAR can query Splunk back for context."

31.8 Playbooks in depth

A playbook is a flow of **blocks**. In the **Visual Playbook Editor (VPE)** you drag blocks onto a canvas and connect them. SOAR generates **Python code** behind it, and you can also write code blocks yourself.

Block types

Block	Purpose
Start / End	Entry and exit
Action	Run an app action on an asset (for example, `ip reputation` via VirusTotal)
Decision	If/else branching on a condition
Filter	Select a subset of artifacts/results that meet a condition
Prompt	Ask a human (approve/deny, or a question), with a timeout and a user/role to ask
Format	Build a text string using results (for example, a message or query)
Utility	Helpers such as `add note`, `add comment`, `update artifact`, `pin`
API	Call a SOAR REST API function
Custom function	Your reusable Python code
Code	Free-form Python

Playbook types

- **Automation playbooks:** run automatically on a container (or manually).
- **Input playbooks:** reusable sub-playbooks called by other playbooks, with defined inputs and outputs (modular design). (Terminology may vary by version.)

Data access: datapaths

Playbooks refer to data with **datapaths**:

- `container:id`, `container:name`, `container:severity`
- `artifact:*.cef.sourceAddress` is the source IP of **every** artifact (`*` = all)
- `ip_reputation_1:action_result.data.*.positives` is the "positives" result of an earlier action

Example playbook: suspicious IP triage

```
START
  │
  ▼
[Filter] artifacts that have cef.sourceAddress
  │
  ▼
[Action] ip reputation (VirusTotal asset)
  │
  ▼
[Decision] malicious score > 5 ?
   │ yes                                  │ no
   ▼                                      ▼
[Action] whois ip (context)           [Utility] add note "benign"
   │                                      │
   ▼                                      ▼
[Prompt] "Block IP x.x.x.x ?"          [Utility] set status → closed
   │ approved
   ▼
[Action] block ip (firewall asset)
   │
   ▼
[Action] create ticket (ServiceNow)
   │
   ▼
[Utility] add note + set severity → END
```

What generated code looks like (simplified, for recognition)

python

```
def on_start(container):
    ip_reputation_1(container=container)
    return

def ip_reputation_1(action=None, success=None, container=None, results=None, handle=None,
    parameters = []
    container_artifact_data = phantom.collect2(container=container, datapath=['artifact:*
    for item in container_artifact_data:
        if item[0] is not None:
            parameters.append({'ip': item[0], 'context': {'artifact_id': item[1]}})
    phantom.act("ip reputation", parameters=parameters, name="ip_reputation_1", assets=["
    return

def decision_1(action=None, success=None, container=None, results=None, handle=None, **kwa
    # branch on the positives value from the results
    ...
```

What to notice:

- `phantom.collect2(...)` gathers data using datapaths.
- `phantom.act("ip reputation", parameters=..., assets=[...], callback=...)` runs the action, then the **callback** function continues the flow.
- **Playbook execution is asynchronous.** Actions finish later, and callbacks continue.
- Function signatures and helper names can differ between versions, so treat this as a reading aid, not copy-ready code. In practice you build playbooks with VPE and let it generate the code.

31.9 When playbooks run

- **Automatically** when a container is created or updated and **matches the playbook's label and conditions** (the playbook must be **active**).
- **Manually** by an analyst, from the container's "Run playbook" button.
- **From another playbook** (input playbooks).
- **From Splunk** via the "Run playbook" alert action.
- **Scope:** a playbook can be set to run on **new artifacts only** or on **all artifacts** of the container.

31.10 Use cases (what to say in an interview)

1. Phishing email response

1. Ingest the reported email (mailbox integration).
2. Extract artifacts: sender, URLs, attachment hashes.
3. Reputation lookups: URLs, domains, and hashes (VirusTotal and others).
4. If malicious: search other mailboxes for the same email, delete or quarantine it, and block the sender or domain (with approval).
5. Notify the user, add notes, create or update the ticket, close the case.

2. Malware alert from EDR

Enrich the hash, check who else has it, **isolate the host** (with approval), open a ticket, notify.

3. Brute-force login from an ES notable

Enrich the IP, query Splunk for other activity, check the user in Active Directory, and if the account is compromised, **disable the user** and **reset the session** (with approval).

4. Threat intel enrichment

Auto-enrich every IP/URL/hash artifact with reputation and attach the result, so the L1 sees a verdict without doing the lookup.

5. User offboarding

Disable accounts across several systems in one workflow.

31.11 Where SOAR fits in the incident lifecycle

```
Detect (ES) → Triage → Investigate → Contain → Eradicate → Recover → Lessons learned
                 │           │           │
              SOAR automates enrichment, containment (with approvals), documentation
```

31.12 Measuring value

Metric	Meaning
MTTD	Mean time to detect
MTTR	Mean time to respond (SOAR reduces this)
Automation rate	% of alerts handled without human steps
Analyst hours saved	Time per alert × number of alerts

31.13 Good playbook design principles

1. Start with a **manual process**, write it down, then automate the safe parts.
2. **Investigate first, contain later**, with an **approval prompt** before risky actions.
3. Keep playbooks **small and modular** (input playbooks, custom functions).
4. **Handle failure:** what if the VirusTotal action times out or the API key expires? Add decision branches and notes.
5. **Log everything** (notes, comments) for audit.
6. **Test in a non-production environment** first, with a test label.
7. Use **allowlists** (do not block internal IPs or critical servers).
8. Name and document clearly: purpose, trigger, assets used, owner.

31.14 Hands-on (without a license, plus with one)

Option A: no SOAR access (do this today)

1. **Paper-design a playbook** for “failed logins from one IP”. Write the steps, blocks, decisions, and which action needs approval.

2. **Build the Splunk side** from Module 29 (the brute-force detection) and save it as an alert. In a real setup, the alert action would be "Send to SOAR".
3. In Splunk, simulate the enrichment step with a lookup (`threat_ips.csv`) and a `where`, so you understand the logic that SOAR would automate.
4. **Explain aloud** your design in 2 minutes, as in an interview.

Option B: SOAR Community Edition or a trial (if available to you)

1. Register and install or open it (follow Splunk's current instructions).
2. **Configure an asset:** Apps → pick an app (for example, a reputation app) → Configure new asset → enter credentials → **Test connectivity**. Do this first, because most failures start here.
3. **Create a container with the REST API** (get an authentication token under your automation user's settings):

bash

```
curl -k -H "ph-auth-token: <token>" -X POST https://<soar-host>/rest/container \
  -d '{"name":"Test brute force","label":"events","severity":"medium","description":"la
```

- `-H "ph-auth-token: ..."` is the SOAR API authentication header. `-X POST` creates an object. `-d` is the JSON body.
- Expected: JSON containing `"success": true` and the new container `id`.

bash

```
curl -k -H "ph-auth-token: <token>" -X POST https://<soar-host>/rest/artifact \
  -d '{"container_id": <id>, "name":"src ip", "label":"event", "cef": {"sourceAddress"
```

This adds an artifact with a source IP (`203.0.113.5` is a documentation-only example address).
4. Open the container in the UI, and **run an `ip reputation` action manually** on the IP.
5. Build a simple playbook in the VPE: Action (`ip reputation`) → Decision → Utility (add note). Set it active with the matching label, create a new container, and watch it run.
6. Check the **Activity/Debug** view of the playbook run.

31.15 Troubleshooting

Symptom	Likely causes and checks
Playbook doesn't run	Playbook not **active**; label doesn't match the container's label; scope (new vs all artifacts); container was created before the playbook was active; playbook errors on load
Action fails	Asset credentials expired or wrong; **Test connectivity** on the asset fails; network/firewall/proxy blocks SOAR from the tool; API rate limit; wrong parameter type
Action returns no data	Artifact field is empty or in the wrong CEF field; datapath typo; filter block removed everything
Playbook stuck waiting	A **prompt** is waiting for a human; an action is still running or timed out; callback not reached
Splunk alert not creating a container	Splunk→SOAR connection or auth token wrong; alert action not enabled; field mapping missing; network port blocked
Artifacts missing	Mapping from Splunk fields to CEF is incomplete
Cloud SOAR can't reach an internal tool	**Automation broker** is down or not assigned to the asset
Works manually, fails automated	Different user/permissions context; approval or role missing for prompts
Duplicate containers	Alert fires repeatedly without throttling (fix in Splunk with throttle) or the ingest dedup settings

Debugging steps:

1. Open the container, then the playbook run's **debug / activity log** to see exactly which block failed.
2. Open the failed action's result: the **error message** is usually clear.
3. Test the asset: **Test connectivity**.
4. Check the SOAR server logs under its log directory (on-prem installs commonly use `/opt/phantom/var/log/phantom/`, so verify the path for your version).
5. Re-run the playbook on a test container.

31.16 Security and access in SOAR (brief)

- **Roles and permissions** control who can run playbooks, approve prompts, edit playbooks, or manage assets.
- **Automation users** are service accounts with API tokens used by integrations. Keep tokens secret and rotate them.
- Store credentials only in **asset configurations**, never inside playbook code.
- **Audit trail:** every action is logged.

31.17 SOAR skills employers look for

Skill	Level
Explain SOAR, containers, artifacts, playbooks, assets	L1 awareness
Read a playbook flow and explain it	L1/L2
Design a phishing or brute-force playbook	L2
Build playbooks in VPE, write small Python custom functions	L2 / SOAR engineer
Integrate Splunk ES with SOAR	L2
Troubleshoot failed actions and assets	L2

For a SOC L1/L2 role, **conceptual fluency plus one or two well-explained use cases** is usually enough. Python-heavy playbook development is a specialist path.

31.18 Interview Q&A

- **What is SOAR?** A platform that orchestrates security tools and automates response using playbooks, to cut response time and analyst workload.
- **SIEM vs SOAR?** The SIEM detects and alerts from logs, and SOAR automates the investigation and response to those alerts.
- **What is a container? An artifact?** A container is the case or event record, and artifacts are the evidence items inside it (IPs, URLs, hashes, users).
- **App vs asset?** An app is the integration for a tool; an asset is a configured instance of it (credentials, URL).
- **What is a playbook?** An automated workflow of actions, decisions, filters, and prompts run against a container.
- **Investigate vs contain actions?** Investigate is read-only enrichment, and contain changes things (block, isolate, disable) and usually needs approval.
- **What is CEF?** The standard field naming (like `sourceAddress`) used on artifacts so actions know where to find data.
- **How does Splunk ES send alerts to SOAR?** Through the SOAR add-on/export app and an adaptive response action ("Send to SOAR" or "Run playbook"), which creates a container with artifacts.
- **Why a human approval prompt?** Containment can cause outages, so a person confirms high-impact actions.
- **A playbook isn't running. What do you check?** Active status, label match, scope, container timing, and the debug log.
- **Walk me through a phishing playbook.** Ingest email, extract artifacts, check reputation, decide, search and quarantine similar emails, block the sender (with approval), notify, ticket, close.
- **What does the automation broker do?** In Cloud SOAR, it lets the cloud instance reach tools inside your private network.
- **How do you measure SOAR success?** MTTR, automation rate, and analyst hours saved.

Common mistake in interviews: claiming SOAR "replaces" analysts. The right answer is that it removes repetitive work so analysts handle the judgment calls.

31.19 Revision (memorize)

- SOAR = orchestrate + automate + respond; ES detects, SOAR responds.
- **Container** (case) → **Artifacts** (evidence, in CEF fields).
- **App** → **Asset** (configured) → **Action** (investigate / contain / correct / generic).
- **Playbook** = blocks: Action, Decision, Filter, Prompt, Format, Utility, Custom function.
- Datapaths: `artifact:*.cef.sourceAddress`.
- Runs on label match when active; also manual or from Splunk.
- Investigate automatically, contain with approval.
- Splunk to SOAR: "Send to SOAR" action; SOAR to Splunk: the Splunk app's `run query`.
- Troubleshoot: asset connectivity, label/active, debug log, prompts waiting.
- Cloud: the automation broker reaches internal tools.

✅ Self-Check (no peeking)

1. What is the difference between a container and an artifact?
2. App vs asset?
3. Which action type should usually require approval, and why?
4. What decides which playbooks run automatically on a new container?
5. What is `artifact:*.cef.sourceAddress`?
6. SIEM vs SOAR in one sentence.

🧪 Exercises

1. **Design on paper:** a playbook for "user reports a phishing email". List the blocks in order, name the actions you'd use, and mark which step needs a prompt.
2. **Design:** a playbook triggered by the Module 29 brute-force alert. Include enrichment, a decision, an approval, and a containment action.
3. **Troubleshoot:** "My playbook is active but nothing happens when a new container arrives." Give your first four checks.
4. **Explain** in 2 minutes: "How does a Splunk ES notable turn into a blocked IP?"